FleetX: A Modular-Monolith Architecture for Intelligent Fleet and Logistics Management with Simulated Real-Time Tracking

Md Ziaul Mustafa
Roll No. 24EARAD100
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

Jitendra Kumar Paswan
Roll No. 24EARAD071
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

Mayank Kumar
Roll No. 24EARAD098
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

Kushagra Gautam
Roll No. 24EARAD087
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

Md Amir Alam
Roll No. 24EARAD099
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

Er. Ram Babu Buri
Supervisor, Assistant Professor
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

***Abstract*—**Fleet and logistics operators must coordinate vehicles, drivers, trips, shipments and live location data, yet small teams and academic projects often adopt microservice architectures whose operational overhead outweighs their benefits. This paper presents FleetX, an Intelligent Fleet and Logistics Management System implemented as a strict modular monolith using Java 21, Spring Boot 3.5.9, MySQL 8.4 LTS, and a React 18 and TypeScript single-page frontend. The system is divided into five domain modules (identity, fleet, drivers and trips, shipments, and tracking) that communicate only through declared contract interfaces and immutable data-transfer records, verified by automated architectural tests. Database evolution is governed by Flyway migrations under reserved per-module version ranges (V100 to V501), with Hibernate schema generation set to validation only. Security relies on server-side sessions, BCrypt hashing, CSRF protection and five canonical roles. A simulated GPS engine interpolates positions along Haversine-based route waypoints and streams telemetry over STOMP WebSockets to a Leaflet map and a stylized Three.js fleet view; all such data is explicitly labelled as simulated. Verification combines 79 backend tests, 34 frontend tests in 7 suites, a 10-step HTTP smoke test, and a MySQL persistence script that checks data survival across restarts. We describe the design rationale, implementation, verification and limitations, and argue that a contract-driven modular monolith is a pragmatic architecture for team-built logistics platforms.

Index Terms*—*fleet management, logistics, modular monolith, Spring Boot, Flyway, STOMP WebSocket, Haversine distance, GPS simulation, role-based access control, React.

I. INTRODUCTION AND PROBLEM STATEMENT

Modern logistics depends on timely knowledge of where vehicles are, which drivers are assigned to which trips, and whether shipments will arrive as promised. Commercial fleet platforms integrate these concerns, but building a comparable system as a student or small-team project raises three recurring problems.

First, *architectural overreach*: microservices are often chosen by default, introducing service registries, message brokers and distributed-data consistency problems that a five-person team cannot realistically operate or debug [1]. Second, *boundary erosion*: even in a single deployable, developers working in parallel tend to reach into each other's repositories and entities, producing tight coupling that defeats modularity [2]. Third, *unmanaged schema drift*: when several developers evolve one relational schema simultaneously, automatic ORM schema generation and ad hoc scripts lead to conflicts and irreproducible environments.

A further difficulty is that real GPS hardware is rarely available in an academic setting, yet tracking features must still be demonstrated and tested without misleading users about data provenance.

FleetX addresses these problems with a modular monolith whose boundaries are enforced by explicit contracts and tests, a migration-governed MySQL schema with ownership ranges, a conventional session-based security model, and a clearly disclosed GPS simulation. Section II states objectives, Section III reviews related work, Section IV defines requirements, Section V describes the architecture, Section VI the implementation, Section VII verification, Section VIII results, and Sections IX–XI limitations, future work and conclusions.

The contributions of this paper are:

1. A contract-driven modular-monolith architecture for a fleet and logistics system with five isolated domain modules.
2. A governance model combining Flyway version-range ownership, validate-only schema handling and architectural tests.
3. A disclosed, simulated real-time tracking pipeline using STOMP WebSockets, Leaflet and Three.js.
4. A layered verification approach (unit, integration, smoke and persistence tests) and a candid account of its limitations.

II. PROJECT OBJECTIVES

The objectives of FleetX are:

1. *One deployable artifact:* build the complete system as a single Spring Boot application on Java 21, avoiding distributed-systems infrastructure.
2. *Module isolation:* partition the domain into five modules and permit interaction only through contract interfaces and immutable records.
3. *Governed schema:* manage all database changes through Flyway with non-overlapping version ranges and `ddl-auto=validate`.

4. *Secure access:* provide role-based access using server-side sessions, BCrypt hashing and mandatory CSRF defense.
5. *End-to-end workflow:* cover vehicle registration, fuel and maintenance, driver onboarding, trip dispatch, shipments, route planning and live monitoring.
6. *Honest tracking demonstration:* provide simulated GPS telemetry delivered in real time, with clear labelling of simulated data and estimated distances.
7. *Reproducible verification:* supply automated tests, a smoke test and a persistence check runnable against a containerized database.

III. RELATED WORK AND TECHNICAL BACKGROUND

A. Monolith, Modular Monolith and Microservices

Parnas argues that modules should be defined by hidden design decisions and narrow interfaces [2]. Microservice literature extends this idea to independently deployable services but acknowledges substantial operational cost, including service discovery, distributed tracing and eventual consistency [1]. A modular monolith keeps the information-hiding discipline while retaining single-process deployment. FleetX adopts this middle path and expresses it through package structure and contract interfaces, in the spirit of the service-layer patterns described by Fowler [3].

B. Spring Boot, Security and Messaging

Spring Boot supplies dependency injection, Spring Data JPA, Spring Security and WebSocket support [4]. Spring Security provides server-side session management, BCrypt encoding and CSRF protection, following OWASP guidance [5]. For push communication, FleetX uses the STOMP sub-protocol [6] over WebSocket with SockJS fallback, served by Spring's in-process broker rather than an external message broker.

C. Schema Migration

Flyway applies versioned SQL migrations in order and records them in a history table, making database state reproducible [7]. Combined with Hibernate's `validate` mode, which only checks that entities match the existing schema, this prevents the ORM from silently altering tables.

D. Geospatial Computation and Visualization

The haversine formula computes great-circle distance between two latitude/longitude pairs and is well behaved for short distances [8]. It yields straight-line surface distance rather than road distance, so FleetX labels derived values as estimates. Leaflet with OpenStreetMap tiles renders 2D maps [9], and Three.js renders a stylized 3D scene [10].

E. Positioning

Commercial telematics products rely on hardware trackers and proprietary back ends. FleetX does not compete with them; it contributes an openly documented, rule-governed reference architecture showing how a team can build a coherent fleet platform with strict boundaries, verified persistence and honest simulation.

IV. SYSTEM REQUIREMENTS AND SCOPE

A. Functional Requirements

Five functional areas are each owned by one module:

- *Identity and administration:* login, logout, current-user lookup, CSRF bootstrap and an administrator user directory.
- *Fleet and assets:* vehicle registration, fuel logs with computed efficiency (km/L), and preventive and upcoming maintenance records.
- *Drivers and trips:* driver onboarding (licence number, category, expiry, experience), trip lifecycle and driver performance.
- *Shipments and routes:* manifests, route waypoints with Haversine distance and estimated arrival time, and tracking events.
- *Tracking and control tower:* simulated GPS sessions, STOMP telemetry, circular and polygonal geofences, alerts, a live map, a 3D fleet view and CSV reports.

B. Non-Functional Requirements

TABLE I
NON-FUNCTIONAL REQUIREMENTS

Concern	Requirement
Deployment	Single modular-monolith artifact; Java 21; Spring Boot 3.5.x
Database	MySQL 8.4 LTS, InnoDB, utf8mb4; BIGINT AUTO_INCREMENT keys; DATETIME(6) timestamps
Schema	Flyway only; ddl-auto=validate
Security	HttpOnly, SameSite=Lax session cookies; BCrypt strength 12; CSRF on state-changing requests
API	/api/v1/ prefix; ISO 8601 UTC timestamps; standard response envelopes
Quality	Conventional Commits; reviewed pull requests; tests pass before merge

C. Roles and Scope

Five canonical roles govern authorization: ADMIN, FLEET_MANAGER, DISPATCHER, DRIVER and VIEWER. In scope are the five modules, a containerized database and a development demo dataset. Out of scope are real GPS hardware, road-network routing, distributed deployment and stateless token authentication.

V. SYSTEM ARCHITECTURE

A. Overall Structure

FleetX is a monorepo with a Spring Boot backend, a React frontend and project documentation. The backend contains five domain packages (`auth`, `fleet`, `drivers`, `shipments`, `tracking`) and a `shared` package holding contracts, DTOs, fallback beans, security configuration and the error envelope. The frontend mirrors this structure through independently owned feature folders. The entry class enables scheduling, which supports the periodic simulation engine. Fig. 1 shows the overall structure.

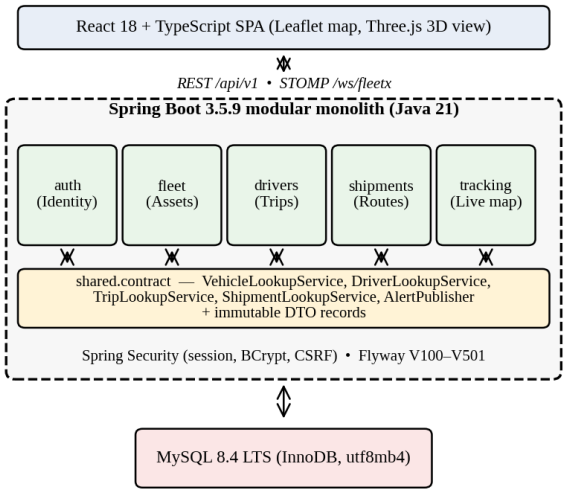


Fig. 1. Modular-monolith architecture of FleetX.

B. Contract Inversion

A domain module must never import or inject another module's repository or entity. All cross-module lookups use five interfaces: `VehicleLookupService` (fleet), `DriverLookupService` and `TripLookupService` (drivers), `ShipmentLookupService` (shipments) and `AlertPublisher` (tracking). Data crossing a boundary is carried by immutable Java records. For example, the shipment service validates a trip through a lookup contract and never touches trip persistence. Each module can thus be developed, tested and replaced behind a stable interface, and later extraction into separate services stays feasible without being committed to now.

C. Data Architecture

Each module owns a reserved Flyway version range (Table II), preventing version collisions in parallel development and making every migration traceable to an owner.

TABLE II
FLYWAY RANGES BY MODULE

No.	Domain	Range
1	Identity and administration	V100–V199
2	Fleet and assets	V200–V299
3	Driver and trip operations	V300–V399
4	Shipments and routes	V400–V499
5	Tracking and control tower	V500–V599

The database runs as a MySQL 8.4 container defined in Docker Compose with a named persistent volume, the utf8mb4 character set and a `mysqladmin ping` health check. Compose is used only for the database, consistent with the rule against distributed service networks.

D. Security Architecture

Authentication uses stateful server-side sessions (JSESSIONID) with HttpOnly and SameSite=Lax attributes. Passwords are hashed with BCrypt at strength 12 and never logged or returned. CSRF protection stays enabled for POST, PUT, PATCH and DELETE: the client calls a CSRF bootstrap endpoint, receives an XSRF-TOKEN cookie and forwards it in the X-XSRF-TOKEN header. Sessions were preferred to JWTs because they allow immediate invalidation at logout, which the smoke test verifies.

E. API Conventions

All endpoints begin with `/api/v1/`. Responses use three envelopes: ApiResponse for single objects, PageResponse for paginated collections and ApiErrorResponse carrying an error code, message and validation errors. Timestamps are ISO 8601 UTC; business codes follow TRIP-YYYY-NNNN and SHP-YYYY-NNNN.

F. Real-Time Tracking Pipeline

The tracking module hosts a simulated GPS service that interpolates vehicle positions along a shipment's route waypoints at a scheduled interval and publishes telemetry to STOMP topics through the in-process broker at `/ws/fleetx`. Subscribers include the Leaflet live map and the Three.js fleet view. Geofence evaluation can raise alerts through the AlertPublisher contract, so other modules emit alerts without depending on tracking internals. Every telemetry payload and UI badge carries the label SIMULATED DATA.

VI. METHODOLOGY AND IMPLEMENTATION

A. Technology Stack

The backend uses Java 21 (records, pattern matching, sealed classes, text blocks), Spring Boot 3.5.9, Spring Data JPA, Spring Security, Spring WebSocket, Flyway and Maven. The frontend uses React 18, Vite, TypeScript, Bootstrap 5.3, Leaflet 1.9 and Three.js r160+, with Axios forwarding CSRF tokens; Vite proxies `/api` and `/ws` to the backend in development.

B. Engineering Governance

Development followed a written rules document binding both human developers and AI coding agents [12]. Its main clauses are: no microservice infrastructure; no cross-module repository leaks; mandatory contract interfaces; Flyway-only schema changes within owned ranges; pinned GA releases; server-side sessions; BCrypt; CSRF; standard envelopes; Conventional Commits; per-module feature branches; two approvals per pull request; and passing tests before merge. Treating rules as an explicit contract gave the team a checkable standard when using automated coding assistants.

C. Phased Development

The project progressed from foundation and per-module development to a final phase of integration, end-to-end verification and stabilization. Each owner delivered services, migrations, REST endpoints, frontend features and tests inside the module's package and Flyway range. Integration then connected modules only through contract interfaces, with fallback beans in the shared package keeping modules runnable while a neighbour was incomplete.

D. Route Planning and Estimation

Given consecutive waypoints, the shipments module computes leg distances with the haversine formula [8]:

$$d = 2R \cdot \arcsin\left(\sqrt{\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cos\varphi_2 \sin^2(\Delta\lambda/2)}\right) \quad (1)$$

where R is Earth's mean radius, φ latitude and λ longitude. Total route distance is the sum of leg distances, and estimated arrival time derives from an assumed average speed. Outputs are labelled Estimated Straight-Line (Haversine) Distance and Estimated Arrival Time because the method ignores road geometry, traffic and stops.

E. Demo Data Seeding

Under the development profile, ordered, idempotent seeders populate all modules through the real domain services and shared contracts rather than raw SQL. The dataset has five accounts (one per role), three vehicles, fuel logs with computed efficiency, maintenance records, two drivers, one in-progress and one planned trip, two shipments (one with three route waypoints), two geofences and an initialized simulated GPS session. Seeding is disabled in production and test profiles.

F. Containerized Deployment

The repository includes a Dockerfile and `.dockerignore`. The smoke test targets a single-origin deployment in which the backend serves both the API and the single-page application, including pushState fallback for client-side routes. Database settings come from an `.env` file derived from a template, with development defaults in the Compose file.

VII. TESTING AND VERIFICATION

Verification is layered so that each level catches a different class of fault.

A. Backend and Frontend Automated Tests

The Maven wrapper's test goal runs 79 automated tests: unit tests, per-module integration and security tests, Flyway schema validation, architectural boundary checks that fail on cross-module repository or entity imports, and full-stack end-to-end workflow tests. Vitest and React Testing Library provide 34 frontend tests in 7 suites covering feature areas and shared components, including the CSRF-aware client.

B. HTTP Smoke Test

A Bash script performs ten sequential checks against a running instance using curl and a cookie jar (Table III). It exits on first failure, making it suitable for CI or post-deployment gating.

TABLE III
SMOKE TEST CHECKS

#	Check	Expected
1	Health endpoint	200
2	Single-origin root	200
3	SPA fallback route	200
4	Unauthenticated fleet query	401
5	CSRF token cookie issued	Present
6	Admin login with CSRF header	200
7	Current identity is ADMIN	Role found
8	Admin user API	200
9	Authenticated fleet query	200
10	Logout, then fleet query	401

C. MySQL Persistence Verification

A Python script using the requests library checks that data written through the API survives a restart. Its create phase authenticates with CSRF, registers a vehicle, a driver, a trip and a shipment in dependency order, and stores the identifiers in a temporary file. After the application and database restart, its verify phase logs in again and fetches each record, asserting key fields such as licence plate, VIN, licence number, trip origin and shipment weight. Creation tolerates a prior run by looking up an existing vehicle, and assertions accept the allowed vehicle statuses, making the check re-runnable. Working over REST, it exercises the same path as real clients, including session, CSRF and the migration-created schema.

D. Process Gates

Pull requests must pass the backend suite, frontend tests and lint, with no stubbed bypasses or disabled security tests, and require two approvals.

VIII. RESULTS AND DISCUSSION

A. Outcomes

The final integrated system reached the stabilized state reported in the project documentation: all five modules operate together in one deployable, migrations V100 to V501 apply cleanly under validation-only Hibernate, and the layered verification suite passes. The demo dataset lets a reviewer log in under any of the five roles and immediately see vehicles, trips, shipments, geofences and a running simulated GPS session without manual data entry.

B. Design Decisions

Boundary enforcement by test. Documented rules erode under deadline pressure; encoding the no-cross-import rule as an architectural test turned it into a build failure, a stronger guarantee than code review alone.

Contracts as integration surface. With five narrow interfaces and immutable records, parallel work needed little coordination beyond agreeing on those interfaces, and fallback beans reduced late integration risk.

Migration ranges. Reserved ranges eliminated version conflicts and made each migration traceable to an owner, at the small cost of non-contiguous numbering.

In-process messaging and honest simulation. Spring's built-in STOMP broker sufficed for the simulated load without external broker operations, and labelling simulated data and estimated distances keeps the demonstration credible.

C. Threats to Validity

The results are functional verification outcomes, not performance benchmarks. No load, latency or concurrency measurements were collected and the simulation uses synthetic data, so no claims about scalability or routing accuracy can be drawn.

IX. LIMITATIONS

1. *Simulated telemetry only:* no physical GPS or telematics feeds are integrated.
2. *Straight-line distance:* Haversine estimates ignore roads, traffic and legal routing constraints.

3. *Single-node scalability:* the in-process broker and server-side sessions live in one instance; scaling out needs shared sessions and an external broker.
4. *No performance evidence:* the suite establishes correctness, not throughput or latency.
5. *Stylized 3D view:* the Three.js scene is procedural, not geometrically accurate.
6. *Artifact inconsistencies:* the rules document names the contract package `com.fleetx.common.contract` while the README uses `com.fleetx.shared.contract`, and the smoke test's default admin password differs from the development seed password. These should be reconciled.
7. *Development credentials:* demo accounts must stay disabled in production profiles.

X. FUTURE WORK

- Integrate real GPS or OBD-II telematics behind the existing tracking contracts.
- Adopt a road-network routing engine for traffic-aware distance and arrival estimates.
- Add load testing and observability (metrics, structured logs, tracing).
- Introduce shared session storage and an external broker for multi-instance deployment.
- Add predictive maintenance and driver-risk analytics from fuel, maintenance and trip history.
- Run the smoke and persistence checks automatically in CI against a containerized stack.
- Assess extracting a module into a service, using the contract interfaces as the seam, if scale demands it.

XI. CONCLUSION

FleetX demonstrates that a fleet and logistics platform with identity, asset, trip, shipment and tracking capabilities can be delivered by a small team as a disciplined modular monolith. Enforcing boundaries through contract interfaces and architectural tests, governing the schema through Flyway with owned version ranges, adopting conventional session-based security, and verifying behaviour through unit, integration, smoke and persistence tests produced a coherent, reproducible system. Clear disclosure of simulated telemetry and estimated routing keeps its claims honest. While performance and real-world data integration remain future work, the architecture offers a practical reference for teams seeking maintainability without the operational cost of distributed services.

ACKNOWLEDGMENT

The authors thank their project guide Er. Ram Babu Buri, Assistant Professor, Department of Artificial Intelligence & Data Science, and Arya College of Engineering & IT for guidance and support during the 2026–27 capstone project.

REFERENCES

[1] S. Newman, *Building Microservices*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[2] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Commun. ACM*, vol. 15, no. 12, pp. 1053–1058, Dec. 1972.

[3] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[4] VMware/Broadcom, "Spring Boot Reference Documentation," 3.5.x. [Online]. Available: https://docs.spring.io/spring-boot/

[5] OWASP Foundation, "Cross-Site Request Forgery Prevention Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/

[6] STOMP Project, "STOMP Protocol Specification, Version 1.2." [Online]. Available: https://stomp.github.io/stomp-specification-1.2.html

[7] Redgate, "Flyway Documentation." [Online]. Available: https://documentation.red-gate.com/flyway

[8] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[9] V. Agafonkin, "Leaflet: an open-source JavaScript library for interactive maps." [Online]. Available: https://leafletjs.com/

[10] R. Cabello et al., "three.js: JavaScript 3D library." [Online]. Available: https://threejs.org/

[11] Oracle, "MySQL 8.4 Reference Manual." [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/

[12] FleetX Team, "FLEETX_AGENT_RULES: Agent and Developer Engineering Rules," project repository, 2026.